

Oberon-0语言

唐雅妍23320143

1.1 编写一个正确的Oberon-0源程序

实现了一个计算最大公因式、二分查找和调用最大公因式的oberon-0，见代码 `gcd.obr`。

它涵盖了：

模块结构：

- 模块：`MODULE Sample; ... END Sample.`
- 声明：
 - 常量声明：`CONST N=040;` 表示数组大小，并且使用0开头定义了八进制常量
 - 类型声明：`Node=RECORD ... END;` 表示两个整合字段的复合类型
 - 变量声明：`ARRAY N OF Node` 定义固定大小的数组，并且元素为自定义类型
- 过程声明与调用：
 - 过程声明：`PROCEDURE Gcd(...);` 和 `PROCEDURE BinSearch;` 定义带参数和不带参数的过程
 - VAR 参数：`VAR ans: INTEGER;` 实现引用参数传递
 - 过程调用：`Gcd(x, y, ans);` 体现参数传递和过程调用
- 语句构造：
 - While和if语句的使用
 - 赋值语句
- 表达式构造：
 - 算术运算、比较运算、逻辑运算均全部涉及，包括`~`，`DIV`，`MOD`，`#`，`&`等等
 - 数组访问：通过数组下标与记录字段访问，如 `a[i].id`
 - 复杂表达式：`(1+r) DIV 2`、`~(i >= n) & n<N` 等
- `write`、`writeln`和`read`的输入输出

1.2 编写上述Oberon-0源程序的变异程序

根据文档定义的异常，对应每一个异常写出对应的变异程序，并且在每一变异程序的第一行用注释，说明该变异产生的错误类型。

```
OberonException
├─ LexicalException
│   ├── IllegalSymbolException
│   ├── IllegalIntegerException
│   ├── IllegalIntegerRangeException
│   ├── IllegalOctalException
│   ├── IllegalIdentifierLengthException
│   └─ MismatchedCommentException
├─ SyntacticException
│   ├── MissingLeftParenthesisException
│   ├── MissingRightParenthesisException
│   ├── MissingOperatorException
│   └─ MissingOperandException
└─ SemanticException
    ├── TypeMismatchedException
    └─ ParameterMismatchedException
```

词法分析：

- 001：有非法@符号
 - 正常写法： `a: ARRAY N OF Node;`
 - 变异写法： `a@: ARRAY N OF Node;`
- 002：0abc非法数字
 - 正常写法： `i := 0;`
 - 变异写法： `i := 0abc;`
- 003：超过了12位（13位）
 - 正常写法： `N=040;`
 - 变异写法： `N=0123456789012;`
- 004：非法八进制（存在98）
 - 正常写法： `N=040;`
 - 变异写法： `N=0409;`
- 005：非法标识符长度（超过了24位）（25位）
 - 正常写法： `a: ARRAY N OF Node;`
 - 变异写法： `arraynofnodearraynofnode1: ARRAY N OF Node;`
- 006：没有匹配注释符号（词法分析）
 - 正常写法： `(* Call the greatest common divisor function.*)`
 - 变异写法： `(* Call the greatest common divisor function.`

语法分析：

- 007: 缺少左括号
 - 正常写法: `Write(a[ans].id)`
 - 变异写法: `Write(a[ans].id`
- 008: 缺少右括号
 - 正常写法: `WHILE (y>0) DO`
 - 变异写法: `WHILE y>0) DO`
- 009: 缺少运算符
 - 正常写法: `t := x MOD y;`
 - 变异写法: `t := x y;`
- 015: 缺少运算符 (预定义函数缺少参数)
 - 正常写法: `Read(x);`
 - 变异写法: `Read();`
- 010: 缺少操作数
 - 正常写法: `i := i + 1`
 - 变异写法: `i := i +`

语义分析:

- 011: 赋值语句将bool类型赋给int类型
 - 正常写法: `ans := x`
 - 变异写法: `ans := (x=2)`
- 012: 算术表达式中int类型与bool类型进行mod运算
 - 正常写法: `t := x MOD y;`
 - 变异写法: `t := x MOD (1<y);`
- 013: 参数传递时期望int类型但传递了bool类型
 - 正常写法: `Gcd(x, y, ans);`
 - 变异写法: `Gcd(b, y, ans);` (注: `b` 为 `BOOLEAN` 类型)
- 014: 函数参数数量不一致
 - 正常写法: `Gcd(x, y, ans);`
 - 变异写法: `Gcd(x, y);`

1.3 讨论Oberon-0 语言的特点

1.3.1 保留字与关键字

保留字包括 VAR, IF、THEN、ELSIF、WHILE等,而**关键字**包括 INTEGER、WRITE、WRITELN等

- 保留字是语法结构词,是语言语法的必要组成部分,用于识别代码的具体结构,不能被重新定义或作为标识符使用,如条件判断模块、循环模块以及定义模块;

- 关键字是标准库提供的预定义名称，实际上和标识符类似，不过是预定义好的，如类型预定义（类似 TYPE 进行定义，如 INTEGER 等），函数预定义（类似 PROCEDURE，如 WRITELN 等），理论上可以被重新定义（但是代码可读性会减弱）。

在 EBNF 定义我们也可以看到，实际上对于关键字在语法识别中是识别为 identifier 来处理，而保留字是直接出现在不同语法里。

1.3.2 与C/C++、JAVA语言对比

1. 规定了赋值运算的结构，对于赋值运算没有返回值，不能实现连续赋值操作，如c中的 `a=b=c`
2. 表达式层次结构较简单，仅支持少量逻辑运算和算数运算，没有C++的位运算、自增自减等复杂运算，因此优先级层数较低
3. 类型较少，与丰富的C++类型不同，该语言只有int和bool类型，不支持浮点数运算，因此减少了强制类型转换操作，且禁止隐式类型转换（如 BOOLEAN 和 INTERGER 不能混用）
4. 没有指针类型等运算
5. 通过对数字长度的约束来省去判断输入数字范围
6. 部分逻辑运算和除法、模运算的符号不同

1.4 讨论Oberon-0文法定义的二义性

1.4.1 表达式

```
expression = simple_expression [( "=" | "#" | "<" | "<=" | ">" | ">=" ) simple_expression];
simple_expression = [ "+" | "-" ] term { ( "+" | "-" | "OR" ) term } ;
term = factor { ( "*" | "DIV" | "MOD" | "&" ) factor } ;
factor = identifier selector | number |
        "(" expression ")" |
        "~" factor ;
```

规定了不同优先级，且统一优先级的为左结合，如term中*/%和与运算均为左结合；而二元加法的优先级比除法低，逻辑判断优先级最低

对于一元运算符+-，simple_expression保证了-2*3被解释为-(2*3)，而-2+3解释为(-2)+3，不会发生歧义，体现的优先级为（从上到下优先级升高）：

1. 逻辑运算等于、大于（大于等于）、不等、小于（小于等于）的优先级最低，且若使用了为语法树的根，并且不能连续使用（如1=2=3，除非用括号）
2. 二元运算符+，-，或运算优先级次低
3. 一元运算+/-较低，他比二元运算符+，-要高但比乘法低

4. 乘法除法模运算和与运算优先级较高，且同时出现时为左结合

5. 括号和~优先级最高

1.4.2 IF-ELSE

对于悬空else的问题，通过 end 解决了 if-if-else 的二义性，传统的 C/C++ 支持不使用括号包裹但若出现 if-if-else 为了实现 else 与最近的 if 配对需要分类设计 matched 和 open 来支持，而该语法强制用户使用 END 包裹，则

- IF expr THEN st IF expr THEN st END ELSE st END 表示 if-(if)-else
- IF expr THEN st IF expr THEN st ELSE st END END 表示 if-(if-else)

```
if_statement = "IF" expression "THEN"
              statement_sequence
              {"ELSIF" expression "THEN" statement_sequence}
              ["ELSE" statement_sequence]
              "END" ;
```

1.5 实验心得

本实验分析了Oberon-0语言的特点，并实现了一个基本包含所有特性的程序和多个异常程序，可以在后续编译器实现作为测试数据。